

Manipulando registradores

- Uso de registradores:
 - | OU (usado para ativar bits)
 - & E (usado para limpar bits)
 - ^ OU EXCLUSIVO (trocar estado de bits)
 - ~ complemento de 1 (1 vira 0, 0 vira 1)(não)
- 0b como prefixo = é uma nomenclatura para indicar que é binário
- É basicamente as portas lógicas de digital
- OU (|): Ligar um bit
 - 0 ou 0: 0
 - 1 ou 0: 1
 - 0 ou 1: 1
 - 1 ou 1: 1
- Para desligar um bit usa &~
- Código de OU:

```
uint_8 status;  
status = 0b00110011;  
status = status | (0b10000000);  
//Vai ligar o primeiro bit, status vira 10110011
```

Status = status | (1 << 4)

- Significa ativar o quarto bit
- Código desligar bit:

```
uint_8 status;  
status = 0b00110011;  
status = status &~ (0b00010000);  
//vai desligar o quarto 1, status vira 00100011
```

- O código acima equivale a:

Status = status &~ (1 << 4)

- Significa desligar o quarto bit

- Deslocamento:

Deslocamento para a esquerda

$y = 1010$
 $x = y \ll 1$
 Resulta
 em: $x = 0100$

um dos 1s foi pro além pois ele deslocou uma casa para a esquerda e não existe essa casa no binário dado

Deslocamento para a direita

$y = 1010$
 $x = y \gg 1$
 Resulta
 em: $x = 0101$

Inatel

Onde era 1 virou 0 e onde era 0 virou 1, porém foi devido ao deslocamento de uma casa

- (1<<2) significa bit 2 ativado
- Tabela do valor dos bits:

valor	128	64	32	16	8	4	2	1
bit	7°	6°	5°	4°	3°	2°	1°	0°

Exercícios

Exercício 1

- Em um valor binário de 8 bits, faça o que se pede:
 - A. Inicie o valor com 127 em decimal;
 - B. 'Ative' apenas o bit mais significativo;
 - C. Desative os 2 menos significativos;
 - D. 'Ative' apenas o bit 5;

A. 01111111

B. $01111111 \mid 10000000 = 11111111$

C. $11111111 \&\sim 00000011 = 11111100$

D. $11111100 \mid 00100000 = 11111100$

```
uint_8 status;  
//A:  
status = 0b01111111;  
//B:  
status = status | (0b10000000);  
//C:  
status = status &\sim (0b00000011);  
//D:  
status = status | (0b00100000);
```

No valor resultante:

A. Teste se o bit 6 está em 1;

B. Teste se o bit 3 está em 0;

C. Inverta o estado de todos os bits;

```
//A:  
if (status & (1 << 6)){  
    printf("Check! O bit 6 está em 1");  
}  
//B:
```

```

if (!(status & (1<<3))){
    printf("Check!0 bit 3 está em Zero");
}
//C:
status = ~(status);

```

- Para testar se um bit está em um local se usa if
- Sintaxe do teste de bit:
 - if (status & (1<<6)) : Isso pega o status (o binário que está dentro da variável status) e pega o binário 01000000, ao usar o registrador & estamos comparando 11111100 com 01000000
 - como funciona:

	11111100	
&	01000000	
Resultado	01000000	

- O resultado 01000000 é diferente de 0 então o C entende como verdadeiro e entra no if
- Ao fazer esse & recebemos 1 apenas se tiver o 1 na posição 6
- Ao usar (!(status & (1<<3))) quer dizer que estamos procurando o zero
- 1 & 1 = 1 (true em C)
- 1 & 0 = 0 (false em C)